



Avsluttende eksamen i
TDT4125 ALGORITMEKONSTRUKSJON

Les hele eksamen før du begynner!

Det kan være lurt å begynne litt på en vanskelig oppgave før du legger den fra deg, så den kan modnes i bakhodet mens du jobber med noe annet.

Disponer tiden, og forbered spørsmål til fagstabens hjelperunde.

Poeng er oppgitt for hver deloppgave. Disse summerer til 120, men maksimal score er 100 poeng. Med andre ord: **En poengsum på over 100 teller som 100.**

Oppgave 1

- a) Forklar kort hvordan kvanteparallellitet virker. Svar gjerne med en ligning. (5 %)

Løsning: En superposisjon av alle basistilstander er $|\phi\rangle = \sum_{i=0}^{2^n-1} c_i|i\rangle$. Om vi anvender en unitær operator A på denne superposisjonen, får vi $A|\phi\rangle = \sum_{i=0}^{2^n-1} c_i A|i\rangle$.

Det finnes en såkalt *effektiv* universell Turing-maskin U , som er slik at for alle $x, \alpha \in \{0, 1\}^*$ så er $U(x, \alpha) = M_\alpha(x)$, der M_α er Turing-maskinen representert av α . Anta at M_α med input x stopper etter T trinn.

- b) Hva kan du si om hvor mange trinn U bruker med input x og α ? (5 %)

Løsning: Den stopper etter $CT \log T$, der C er en konstant som er uavhengig av $|x|$, og kun avhengig av M_α sin alfabetstørrelse, antall tapes og antall tilstander.

(Relevant pensum: Arora & Barak, teorem 1.14)

- c) Du har et lineært program der minst én av to bestemte restriksjoner må holde. Du kan anta at variablene er ikke-negative og begrenset oppad, dvs., $0 \leq x_j \leq u_j$ for hver j . Forklar hvordan du vil håndtere dette med heltallsprogrammering. (10 %)

Løsning: Anta at restriksjonene er $\sum_j a_{1j}x_j \leq b_1$ og $\sum_j a_{2j}x_j \leq b_2$. Innfør to store verdier M_1 og M_2 , der

$$M_1 \geq \max_x \sum_j a_{1j}x_j - b_1 \quad \text{og} \quad M_2 \geq \max_x \sum_j a_{2j}x_j - b_2.$$

Siden x er begrenset, er disse verdiene veldefinerte. Innfør så en binær variabel y , og skriv om restriksjonene til $\sum_j a_{1j}x_j \leq b_1 + M_1y$ og $\sum_j a_{2j}x_j \leq b_2 + M_2(1 - y)$.

(Relevant pensum: Utdelt materiale, *AIMMS Modeling Guide*, avsnitt 7.3)

- d) Forklar kort hva $T_P(n)$ er, og oppgi to nedre grenser for den. Forklar hvorfor disse grensene holder. Ingen utregninger er nødvendig; det holder med en intuitiv forklaring, gjerne også knyttet opp mot beregningsgrafene (*computation dag*). (10 %)

Løsning: Det er kjøretid for problemstørrelse n på P prosessorer. Det totale arbeidet, eller det totale antallet noder i grafen, er $T_1(n)$, og vi kan aldri få noe bedre kjøretid enn når alle prosessorene jobber hele tiden, altså $T_1(n)/P$. Vi kan heller aldri få noe bedre kjøretid enn om vi har uendelig mange prosessorer, altså $T_\infty(n)$, som er lengden på den lengste, eller *kritiske*, stien i beregningsgrafene.

- e) Forklar kort hvordan Euler-tour-teknikken brukes for å finne dybden til alle n noder i et binært tre. Hva blir kjøretiden? (10 %)

Løsning: Alle noder deles i tre noder som kobles sammen til en liste som representerer en traversering av treet. Noder man treffer på vei ned har vekt 1, noder man treffer på vei opp har vekt -1 og de andre har vekt 0. Man bruker så en parallell-prefiks-algoritme for å finne dybdene i logaritmisk tid.

(Relevant pensum: Cormen et al., 1. utg., § 30.1.3)

Oppgave 2

- a) Forklar definisjonene av klassene P, NP, co-NP, NP-hard og NPC. (10 %)

Løsning: Her aksepteres mange forklaringer, så lenge de uttrykker essensielt det samme som de formelle definisjonene i læreboka.

(Relevant pensum: Cormen et al., s. 1059, 1064, 1069)

Dine to medstudenter Lurvik og Smartnes har en opphetet diskusjon, der Lurvik mener 2-CNF-*satisfiability* er NP-komplett, mens Smartnes mener at det ikke er det.

- b) Hva er din mening i saken? Begrunn svaret. (10 %)

Løsning: Det er god grunn til å tro at det ikke er NP-komplett, siden vi kan løse det i polynomisk tid. Men siden vi ikke vet om $P = NP$, så kan vi ikke si det sikkert.

To grafer $G = (V, E)$ og $G' = (V', E')$ er *isomorfe* hvis det finnes en bijeksjon (én-til-én-korrespondanse) $f : V \rightarrow V'$ slik at $(u, v) \in E$ hvis og bare hvis $(f(u), f(v)) \in E'$. La $\text{GRAPH-ISOMORPHISM} = \{\langle G, G' \rangle : G \text{ og } G' \text{ er isomorfe grafer}\}$.

- c) Vis at $\text{GRAPH-ISOMORPHISM} \in \text{NP}$. (10 %)

Løsning: Her kan for eksempel et sertifikat være bijeksjonen f , oppgitt som en liste med par $(u, f(u))$, og vi kan i lineær tid sjekke at kravene til isomorfisme er oppfylt.

(Relevant pensum: Cormen et al., avsnitt 34.2; se oppgave 34.2-1.)

- d) Anta at du har en algoritme A som finner korteste enkle vei (*simple path*) mellom to noder s og t i en komplett, vektet, urettet graf. Anta at A har kjøretid $T(n)$, der n er antall noder. Vis hvordan A kan brukes til å løse *Hamilton-sykel*-problemet (HAM-CYCLE) med kjøretid $T(n) \cdot O(n^k)$, for en eller annen k . Prøv å gjøre k så liten som mulig. Diskuter hva k blir for løsningen din. (10 %)

Løsning: La $G = (V, E)$ være en HAM-CYCLE-instans. Vi lager så en komplett graf $G' = (V, E')$, og for hver $e \in E'$ setter vi $w(e) = -1$ hvis $e \in E$ og $w(e) = 0$ ellers. Dette kan vi gjøre med kjøretid $O(n^2)$. Velg en vilkårlig node $v \in V$, og erstatt denne med to nye noder, s og t , som begge har kanter til naboene til v . Finn så korteste enkle vei fra s til t . Siden dette er en enkel sti, kan den gå innom hver node maksimalt én gang, og kan dermed maksimalt inneholde $V - 1$ kanter, og det skjer hvis den besøker alle nodene. Med andre ord, G har en Hamilton-sykel hvis og bare hvis den korteste enkle veien fra s til t har lengde $-V + 1$.

Så lenge $T(n) = \Omega(n^2)$, kan vi løse problemet med kjøretid $T(n)$, altså med $k = 0$. Dette er høyst sannsynlig, siden vi ellers ville ha $P = NP$. I det ekstremt usannsynlige tilfellet at $T(n) = \omega(n^2)$, så vil vi bruke en høyere verdi for k ($0 < k \leq 2$) så den totale kjøretiden ble kvadratisk.

Du vil holde et selskap for venner og bekjente, men vi ikke ta deg bryet med å invitere alle. I stedet vil du velge ut så få som mulig som tilsammen kjenner alle, og få disse til å spre invitasjonen videre. Boka *Algorithms to Live By* av Christian & Griffiths (2016) foreslår en avrundet kontinuerlig slakking (*relaxation*) av dette problemet, dvs. at du finner en optimal løsning på problemet der alle kan få en fraksjonell invitasjon, og deretter runder du alle invitasjonene til 0 eller 1. Forfatterne sier så:

It turns out that for the invitations problem, continuous relaxation with rounding will give us an easily computed solution that's not half bad: it's mathematically

guaranteed to get everyone you want to the party while sending out at most twice as many invitations as the best solution obtainable by brute force. (s. 177)

Uten å vite akkurat hvilke restriksjoner som settes på de fraksjonelle invitasjonene, kan vi ikke vite hvordan de har kommet frem til denne konklusjonen. Likevel er det mulig å trekke noen generelle konklusjoner.

- e) Diskuter påstandene til Christian & Griffiths. Hvilket problem fra pensum tilsvarer *the invitations problem*? Begrunn svaret relativt grundig. Hva kan du si om approksimering av dette problemet? Kjenner du noen lignende problemer som passer resonnetet deres? (10 %)

Løsning: Det tilsvarer *the set-covering problem*. Dvs., vi kan redusere til og fra dette problemet uten å endre approksimeringsgraden.

Det er opplagt at problemet kan formuleres som et *set-covering*-problem, der elementene er personene, og mengdene representerer hvem de ulike personene kjenner. Men for å vise at problemet tilsvare *set-covering*, må vi redusere den andre veien. Vi starter altså med en instans $(X, \mathcal{F})$ av *set-covering*-problem. Vi lar $X \cup \mathcal{F}$ være mengden med personer. Vi lar alle personer i $\mathcal{F}$ kjenne hverandre, og for alle $x \in X, S \in \mathcal{F}$, hvis $x \in S$ så kjenner x og S hverandre. Vi finner så en mengde med et minimalt antall personer som tilsammen kjenner alle. Hvis en person x i løsningen er hentet fra X , kan vi bytte ut x med x sin eneste venn i $\mathcal{F}$, uten at løsningen blir dårligere. (Dette kan vi gjøre i lineær tid.) Den minimale mengden med personer er da tatt fra $\mathcal{F}$, og tilsvarer dette et minimalt *set cover* for den opprinnelige instansen. Siden løsningene er like store i begge tilfeller (og for reduksjon begge veier) vil en $\alpha(n)$ -approksimasjon for det ene problemet også være en $\alpha(n)$ -approksimasjon for det andre.

(En reduksjon fra problemet DOMINATING-SET er mer direkte, men dette problemet er ikke beskrevet i pensum.)

Vi vet at det finnes en $c > 0$ som er slik at hvis vi kan finne en $c \ln n$ -approksimering for *set-cover*-problem med $\alpha(n) \leq c \ln n$, så er $P = NP$. Med andre ord er påstanden til Christian & Griffiths om en 2-approksimering svært usannsynlig.

Et problem som ligner svært er VERTEX-COVER. Her vil vi ikke dekke alle *personene* men alle *vennskapene* (kantene). Og avrundet kontinuerlig slakking av dette problemet gir (for et passende lineærprogram, for eksempel) en 2-approksimering.

(Relevant pensum: Williamson & Shmoys, teorem 1.14 og avsnitt 1.3)

Oppgave 3

- a) Forklar hva et parametrisert problem er, og hva det vil si at problemet er *fixed-parameter tractable*. (10 %)

Løsning: Et parametrisert problem er et språk $L \subseteq \Sigma^* \times \mathbb{N}$, der Σ er et fast, endelig alfabet. For en instans $(x, k) \in \Sigma^* \times \mathbb{N}$ kalles k parameteren. Problemet er *fixed-parameter tractable* hvis det finnes en algoritme $\mathcal{A}$, en beregnbar funksjon $f : \mathbb{N} \rightarrow \mathbb{N}$ og en konstant c slik at $\mathcal{A}$ avgjør hvorvidt $(x, k) \in L$ med kjøretid begrenset av $f(k) \cdot |(x, k)|^c$.

Her godtas også mindre formelle forklaringer som får frem essensen, spesielt at potensen i kjøretiden ikke avhenger av k .

(Relevant pensum: Cygan et al., defn. 1.1 og 1.2)

- b) Når man skal finne et *edge clique cover* og to noder u og v har samme lukkede nabolag (mengden av naboene og noden selv), hvorfor er det trygt å redusere problemet ved å slette den ene noden? (10 %)

Løsning: De to naboene u og v kan behandles nøyaktig likt i den optimale løsningen, og noder som finnes i nøyaktig de samme klikkene må nødvendigvis ha samme nabolag. Dermed kan en løsning der den ene noden v er slettet utvides ved å legge til v i alle klikkene som inneholder u .

(Relevant pensum: Cygan et al., s. 25)

En *matching* i en urettet graf $G = (V, E)$ er et sett med disjunkte kanter, det vil si en delmengde M av E der ingen kanter i M deler noder. En *maksimal* matching er en som ikke kan utvides med kanter fra E og fortsatt være en matching. Du har oppgitt en urettet graf G og et positivt heltall k , og ønsker å avgjøre om det finnes en maksimal matching i G med k eller færre kanter.

- c) Beskriv en polynomisk kjerne (*kernel*) for problemet, parametrisert av k . Forklar fremgangsmåten og vis utregning. (Du kan beskrive fremgangsmåten med egne ord; du trenger ikke bruke en liste med reduksjonsregler, som i læreboka.) (10 %)

Hint: Hva kan du si om endepunktene til matchingen?

Løsning: Fjern først alle isolerte noder, siden de ikke kan inngå i matchingen.

Nodene i matchingen utgjør et nodedekke (*vertex cover*) av størrelse maks $2k$. Dermed må alle noder med grad større enn $2k$ være et endepunkt i løsningen, ellers vil nodedekket vi ender opp med bli større enn $2k$ (jfr. VC.2), og matchingen må da bli større enn k .

Anta at vi har en ja-instans. La A være nodene med grad større enn $2k$. Siden alle disse må inngå i M , har vi at $|A| \leq 2k$, og det er maksimalt $2k(2k - 1)/2 = 2k^2 - k$ kanter mellom disse nodene.

La $B = G \setminus A$. Alle isolerte noder i B må være tilkoblet A . For hver node i A kan maksimalt én node i B være med i en løsning, og det er vilkårlig hvilken av dem som

velges, så vi sletter alle bortsett fra én. Vi har da maksimalt $2k$ isolerte noder i B etter slettingen.

Hvis M er en maksimal matching må alle kanter i E enten være med i M eller være tilkoblet en av nodene i M . Siden nodene i B har en grad på maksimalt $2k$, betyr det at antall kanter tilknyttet nodene i B er maksimalt $2(2k - 1) \cdot |M| \leq (4k - 1) \cdot k = 4k^2 - k$ med $4k^2$ tilhørende (ikke-isolerte) noder.

Den reduserte grafen inneholder altså maksimalt $2k^2 - k + 4k^2 - k = 6k^2 - 2k$ kanter og $4k^2 + 2k$ noder. Har vi flere noder eller kanter enn dette konkluderer vi med at vi har en nei-instans.

(Relevant pensum: Cygan et al., kap. 2; se oppgave 2.7)